Authly

Sistema de Gestión de Accesos

Documentación Frontend

Arquitectura general, estructura de pantallas, estilos, tema visual y transiciones del panel web

Ingeniería de Software II — UCALP

El frontend de **Authly** constituye el panel de administración web mediante el cual el Administrador de Organización gestiona su cuenta, sus aplicaciones, los roles con permisos y los usuarios registrados. Está desarrollado íntegramente con tecnologías estándar de la web (HTML, CSS y JavaScript puro con módulos ES nativos, sin librerías ni frameworks externos) y se sirve mediante Nginx conectándose a la API REST del backend bajo el prefijo `/api/`.

1. Punto de entrada y redirección (`index.html`)

El archivo `index.html` actúa como puerta de entrada única al sitio web y no posee contenido visual para el usuario. Su propósito es resolver de forma inmediata el destino de navegación y evitar parpadeos visuales al iniciar:

- **Aplicación anticipada del tema:** Incorpora un pequeño script sincrónico en la cabecera que lee la preferencia guardada en el navegador y aplica el tema claro u oscuro antes de dibujar cualquier elemento en pantalla.
- **Enrutamiento según sesión:** Mediante su script asociado (`index.js`), verifica si existe un token de autenticación almacenado localmente. Si hay sesión activa, redirige al listado de aplicaciones (`apps.html`); en caso contrario, envía al formulario de acceso (`login.html`).
- **Navegación limpia:** La redirección se ejecuta reemplazando la entrada actual del historial del navegador (`window.location.replace`), de modo que el botón «Atrás» no devuelva al usuario a una página intermedia vacía.

2. Pantallas del panel web

El sistema sigue una arquitectura multipágina (MPA) dividida en dos grupos funcionales: las pantallas públicas de autenticación y las pantallas privadas del panel de administración.

2.1. Registro de cuenta y organización (`register.html`)

- **Estructura HTML:** Presenta el logotipo institucional, un indicador visual de tres pasos (*Ingreso de datos*, *Verificación* y *Registro completado*) y tres tarjetas secuenciales que se muestran de a una. El primer paso organiza un formulario en dos columnas (datos personales del administrador y datos de la organización con CUIT de 11 dígitos); el segundo contiene el campo para el código numérico de 6 dígitos; y el tercero muestra la confirmación de alta exitosa.
- **CSS específico:** Carga `auth.css` junto con las hojas compartidas, definiendo el contenedor centrado de ancho medio, la grilla en dos columnas para el formulario inicial, el indicador de pasos y los íconos integrados dentro de los campos.
- **JavaScript puntual (`register.js`):** Controla el avance entre pasos, valida localmente los formatos antes de enviar, conserva los datos del primer paso en memoria para reenviarlos junto con el código de verificación, gestiona el botón **«Reenviar código»** y muestra los mensajes de error debajo de cada campo afectado.

2.2. Inicio de sesión (`login.html`)

- **Estructura HTML:** Utiliza una tarjeta centrada angosta con el logotipo, un título de bienvenida, campos para correo electrónico y contraseña (con botón de ojo para alternar su visibilidad), un bloque para errores generales y enlaces secundarios hacia el registro (**«No tengo cuenta»**) y la recuperación de clave (**«Olvidé mi contraseña»**).
- **CSS específico:** Emplea `auth.css` con la variante de contenedor angosto (`.contenedor-angosto`) optimizada para formularios simples de una sola columna.
- **JavaScript puntual (`login.js`):** Verifica al cargar si existe un aviso pendiente de sesión expirada para mostrarlo como notificación emergente, valida que el correo tenga formato correcto y que la contraseña no esté vacía, consulta las credenciales contra la API, guarda los datos de sesión en el navegador y redirige a `apps.html` . Ante credenciales incorrectas, presenta siempre un mensaje genérico fijo sin revelar qué dato falló.

2.3. Recuperación de contraseña (`recovery.html`)

- **Estructura HTML:** Combina la tarjeta angosta del inicio de sesión con el indicador de tres pasos del registro: solicitud mediante correo electrónico en el primer paso; ingreso del código de 6 dígitos junto con la nueva contraseña y su confirmación en el segundo; y pantalla de éxito en el tercero.
- **CSS específico:** Utiliza `auth.css` para mantener la coherencia visual y tipográfica de todo el flujo de autenticación.
- **JavaScript puntual (`recovery.js`):** Avanza siempre al segundo paso tras enviar el correo (manteniendo una respuesta uniforme por privacidad), valida la complejidad de la nueva clave, gestiona la acción **«Volver a pedir un código»** limpiando los campos sensibles y ocultando las contraseñas, y muestra mensajes de error fijos ante códigos inválidos o contraseñas repetidas.

2.4. Gestión de aplicaciones (`apps.html`)

- **Estructura HTML:** Incorpora el encabezado autenticado superior, un bloque de título con el botón **«Crear aplicación»**, un texto resumen con el total de aplicaciones y usuarios de la organización, una tabla paginada (con nombre, descripción, **Client ID** completo con botón de copiado y cantidad de usuarios), un estado vacío con borde punteado cuando aún no hay registros y dos ventanas modales (`<dialog>`): una para el alta de aplicación y otra para exhibir por única vez el **Client Secret** generado.
- **CSS específico:** Carga `panel.css` , que define el encabezado institucional, el contenedor ancho de administración, el estilo de tablas, los cuadros de diálogo y los paneles de credenciales.
- **JavaScript puntual (`apps.js`):** Verifica la sesión activa, solicita el listado paginado al backend, controla la apertura y validación del modal de creación, bloquea el cierre accidental del panel de credenciales hasta que el administrador confirme haber copiado el secreto y actualiza la tabla automáticamente.

2.5. Gestión de aplicación específica (`app.html`)

- **Estructura HTML:** Es la vista más completa del panel. Muestra el encabezado autenticado, un enlace de retorno al listado, la cabecera de la aplicación seleccionada (nombre, descripción y botón **«Editar»**), el bloque de credenciales (**Client ID** copiable y botón **«Regenerar secret»**), un sistema de dos pestañas (**«Roles»** y **«Usuarios»**) con sus respectivas tablas y controles de paginación, y una sección inferior de **Acciones irreversibles** para eliminar la aplicación. Incluye múltiples ventanas modales para editar la aplicación, regenerar el secret, crear o editar roles con filas dinámicas de *recurso + acción*, confirmar cambios en el rol por defecto, editar datos y roles de usuarios, y confirmar eliminaciones.
- **CSS específico:** Utiliza `panel.css` para estilizar las pestañas (`[role="tab"]`), las etiquetas de permisos, las píldoras de rol predeterminado, las filas dinámicas de formulario, las listas de casillas de selección y la franja visual de advertencia en acciones irreversibles.
- **JavaScript puntual (`app.js` y carpeta `js/app/`):** El archivo `app.js` funciona como coordinador principal e inicializa en orden diez módulos especializados dentro de `js/app/` , compartiendo información a través de un objeto de estado común (`estado.js`):
  - *Aplicaciones y credenciales:* `aplicacion.js` , `credenciales.js` , `editar-app.js` y `eliminar-app.js` (exige escribir el nombre de la aplicación para habilitar el borrado y bloquea la acción si existen usuarios activos).
  - *Roles y permisos:* `roles.js` , `formulario-rol.js` y `eliminar-rol.js` (administra pares recurso-acción normalizados en minúsculas y advierte si se quita el rol predeterminado).
  - *Usuarios:* `usuarios.js` (carga diferida al abrir la pestaña por primera vez), `editar-usuario.js` (edición conjunta de nombre, apellido y roles asignados, manteniendo el correo como solo lectura) y `eliminar-usuario.js` .

3. Cuestiones comunes a todas las pantallas

Para evitar duplicación de código y garantizar coherencia visual y funcional, todas las vistas comparten una base modular de hojas de estilo y utilidades JavaScript.

3.1. Hojas de estilo compartidas (CSS)

Archivo CSS	Responsabilidad en el sistema
`variables.css`	Centraliza toda la paleta de colores del sistema mediante variables CSS (`--color-primario`, `--fondo-principal`, `--texto-principal`, `--error`, `--exito`, `--advertencia` y `--peligro`), asegurando contraste adecuado y evitando colores fijos dispersos.
`base.css`	Define el reinicio de estilos del navegador, la tipografía del sistema, la jerarquía de títulos, los contenedores generales, la clase utilitaria `.oculto` y la estructura del botón flotante de cambio de tema.
`components.css`	Estiliza los componentes reutilizables: campos de texto y áreas de texto, botones (principales, secundarios, de peligro y deshabilitados), mensajes de validación, avisos emergentes (*toasts*), el cambio de logotipo según tema, el paginador numérico y los estados de carga.
`transiciones.css`	Contiene las reglas de transición entre páginas y la desactivación global de movimientos cuando el usuario activa la preferencia de accesibilidad correspondiente en su sistema operativo.

Adaptabilidad móvil (Responsive): Todas las pantallas siguen un diseño adaptable con un punto de corte principal en **640 px** y ajustes específicos para pantallas bajas. En dispositivos móviles, el encabezado se reorganiza en dos filas, los botones de formularios se apilan al ancho completo y las tablas de aplicaciones, roles y usuarios se transforman automáticamente en tarjetas individuales que exhiben el rótulo de cada dato.

3.2. Módulos JavaScript compartidos (`js/modules/`)

Módulo JS	Funcionalidad transversal
`api.js`	Cliente HTTP centralizado. Encapsula todas las peticiones al backend bajo `/api/`, adjunta automáticamente el token de autorización en rutas protegidas, traduce nombres de campos entre formatos del servidor y del panel, e intercepta respuestas de sesión vencida (código 401).
`auth.js`	Administra el ciclo de vida de la sesión en `localStorage` (guardar, consultar y limpiar token, correo y organización), protege las vistas privadas mediante `exigirSesion()` (incluso al restaurar páginas con el botón «Atrás» del navegador) y ejecuta el cierre por expiración.
`validators.js`	Implementa las validaciones previas en el cliente (formato de correo, complejidad de contraseña, longitud de CUIT y códigos de 6 dígitos, campos obligatorios y detección de permisos duplicados) antes de enviar datos al servidor.

Módulo JS	Funcionalidad transversal
`notifications.js`	Gestiona la presentación de errores junto a los campos, las notificaciones emergentes temporizadas en la esquina superior derecha y el almacenamiento de avisos en `sessionStorage` para que sobrevivan a una redirección entre páginas.
`components.js`	Provee ayudantes interactivos comunes: navegación entre pasos (`irAlPaso`), copiado al portapapeles con confirmación visual (`activarCopia`), alternancia de visibilidad en contraseñas (`activarOjo`), control de pestañas, paginador numérico y estados de carga en botones.
`encabezado.js` y `utils.js`	El primero renderiza los datos de organización y correo junto al botón **«Cerrar sesión»** en las vistas autenticadas; el segundo ofrece lectura de parámetros de URL, normalización de textos y concordancia gramatical de cantidades.
`theme.js`	Controla la lectura, persistencia y alternancia entre el modo claro y el modo oscuro en todas las vistas.

4. Cambio de tema claro y oscuro

El panel ofrece soporte completo para **modo claro** (predeterminado) y **modo oscuro** en la totalidad de sus pantallas y cuadros de diálogo:

- **Mecanismo basado en atributos y variables:** El modo oscuro se activa aplicando el atributo `data-tema="oscuro"` sobre la etiqueta raíz `<html>`. En `variables.css`, este selector redefine los valores de fondos, bordes, textos y tonos de acción (por ejemplo, adaptando el azul institucional `#082A4D` a tonos más luminosos sobre fondos oscuros) sin alterar la estructura de las reglas CSS.
- **Persistencia y prevención de parpadeos:** La elección del usuario se guarda en `localStorage` bajo la clave `tema`. Cada documento HTML ejecuta un breve script sincrónico al inicio del `<head>` para aplicar el atributo antes del primer pintado, evitando que la página destelle en blanco al navegar en modo oscuro.
- **Botón de alternancia y cambio de logotipo:** Todas las pantallas incluyen en la esquina superior derecha un botón accesible cuyo `aria-label` se actualiza dinámicamente. En su interior superpone dos íconos SVG (luna y sol) que rotan y se desvanecen de forma cruzada al cambiar de estado. Asimismo, el CSS alterna automáticamente entre la versión a color y la versión blanca del logotipo de Authly.

5. Animaciones, transiciones y accesibilidad visual

Con el objetivo de brindar una experiencia fluida y retroalimentación visual clara sin comprometer el rendimiento ni la sobriedad del diseño, el frontend incorpora microinteracciones controladas por CSS y APIs nativas del navegador:

- **Transición entre páginas:** Mediante la regla CSS `@view-transition { navigation: auto; }` en `transiciones.css`, el navegador aplica un desvanecido suave al navegar entre documentos HTML. El encabezado autenticado posee un nombre de transición propio para mantenerse estable durante el cambio de vista.
- **Transiciones dentro de la misma página:** El ayudante `conTransicion()` utiliza `document.startViewTransition()` (con respaldo directo en navegadores sin soporte) para suavizar el cambio de tema visual y la alternancia entre las pestañas **«Roles»** y **«Usuarios»**.
- **Apertura y cierre de ventanas modales:** Los elementos `<dialog>` emplean `@starting-style` junto con transiciones discretas (`allow-discrete`) para escalar y aparecer suavemente tanto el cuadro de diálogo como su fondo oscurecido (`::backdrop`).
- **Notificaciones y errores interactivos:** Los mensajes emergentes (*toasts*) ingresan deslizándose desde el margen derecho y se desvanecen al cerrarse. Por su parte, los mensajes de error en formularios ejecutan una animación breve de temblor horizontal cada vez que se activan para captar la atención sobre el campo inválido.
- **Tablas y botones en proceso:** Al cargar listados, las filas de las tablas aparecen con un desvanecido escalonado mediante reglas CSS. Cuando un formulario ejecuta una operación asincrónica, el botón se deshabilita temporalmente, cambia su etiqueta y muestra un indicador giratorio (*spinner*) junto al texto hasta recibir respuesta.

> **Respeto por las preferencias de accesibilidad**
>
> Todas las animaciones y transiciones del sistema incluyen consultas de medios `@media (prefers-reduced-motion: reduce)` que desactivan movimientos, desplazamientos y escalados cuando el usuario tiene configurada la reducción de movimiento en su dispositivo.